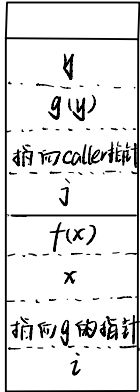


练习 7.2.4: 下面是两个 C 语言函数 f 和 g 的概述:

```
int f(int x) { int i; ... return i+1; ... }
int g(int y) { int j; ... f(j+1) ... }
```

也就是说, 函数 g 调用 f 。画出在 g 调用 f 而 f 即将返回时, 运行时栈中从 g 的活动记录开始的顶端部分。你可以只考虑返回值、参数、控制链以及存放局部数据的空间。你不用考虑存放的机器状态, 也不用考虑没有在代码中显示的局部值和临时值。但是你应该指出:

- 1) 哪个函数在栈中为各个元素创建了所使用的空间?
- 2) 哪个函数写入了各个元素的值?
- 3) 这些元素属于哪个活动记录?



1) $g: f(x), x$
 f : 指向 g 的指针, i

2) $g: x$
 f : 指向 g 的指针, $i, f(x)$

3) $g: y, g(y)$, 指向 caller 的指针, j
 $f: f(x), x$, 指向 g 的指针, i

High
↑
Low

练习 7.2.5: 在一个通过引用传递参数的语言中, 有一个函数 $f(x, y)$ 完成下面的计算:

```
x = x + 1; y = y + 2; return x+y;
```

如果将 a 赋值为 3, 然后调用 $f(a, a)$, 那么返回值是什么? 12

练习 7.2.6: C 语言函数 f 的定义如下:

```
int f(int x, *py, **ppz) {
    **ppz += 1; *py += 2; x += 3; return x+*py+**ppz;
}
```

变量 a 是一个指向 b 的指针; 变量 b 是一个指向 c 的指针, 而 c 是一个当前值为 4 的整数变量。

如果我们调用 $f(c, b, a)$, 返回值是什么? 21

练习 7.3.1: 图 7-15 中给出了一个按照非标准方式计算 Fibonacci 数的 ML 语言的函数 `main`。函数 `fib0` 将计算第 n 个 Fibonacci 数 ($n \geq 0$)。嵌套在 `fib0` 中的是 `fib1`, 它假设 $n \geq 2$ 并计算第 n 个 Fibonacci 数。嵌套在 `fib1` 中的是 `fib2`, 它假设 $n \geq 4$ 。请注意, `fib1` 和 `fib2` 都不需要检查基本情况。我们考虑从对 `main` 的调用开始, 直到 (对 `fib0(1)` 的) 第一次调用即将返回的时段, 请描述出当时的活动记录栈, 并给出栈中的各个活动记录的访问链。

```
fun main () {
  let
    fun fib0(n) =
      let
        fun fib1(n) =
          let
            fun fib2(n) = fib1(n-1) + fib1(n-2)
          in
            if n >= 4 then fib2(n)
            else fib0(n-1) + fib0(n-2)
          end
        in
          if n >= 2 then fib1(n)
          else 1
        end
      in
        fib0(4)
      end;
end;
```

图 7-15 计算 Fibonacci 数的嵌套函数

